

Patterns found in a code example

trap_omp.c

```
double start = omp_get_wtime();
#pragma omp parallel for \
default(none) \
private(i) \
shared(n, a, h) \
reduction(+:integral)
for(i = 1; i < n; i++) {
    integral += f(a+i*h);
}

integral = integral * h;

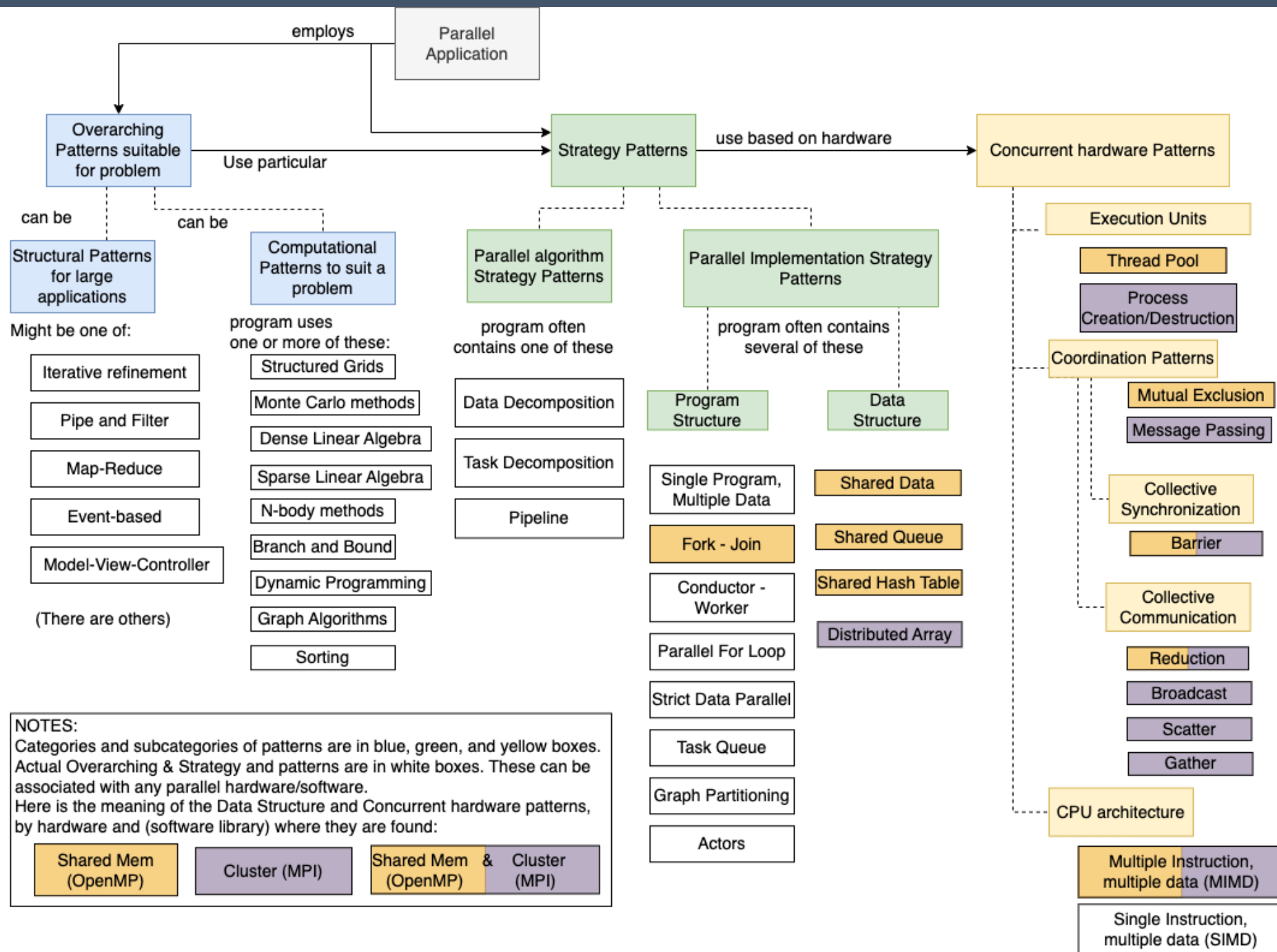
// Measuring the elapsed time
double end = omp_get_wtime();
// Time calculation (in seconds)
double elapsed_time = end -
start;

printf("%lf\t", elapsed_time);
```

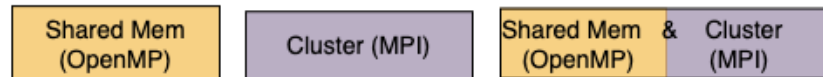
trap-omp example:

What patterns are used?

The patterns framework



NOTES:
 Categories and subcategories of patterns are in blue, green, and yellow boxes. Actual Overarching & Strategy and patterns are in white boxes. These can be associated with any parallel hardware/software. Here is the meaning of the Data Structure and Concurrent hardware patterns, by hardware and (software library) where they are found:



```

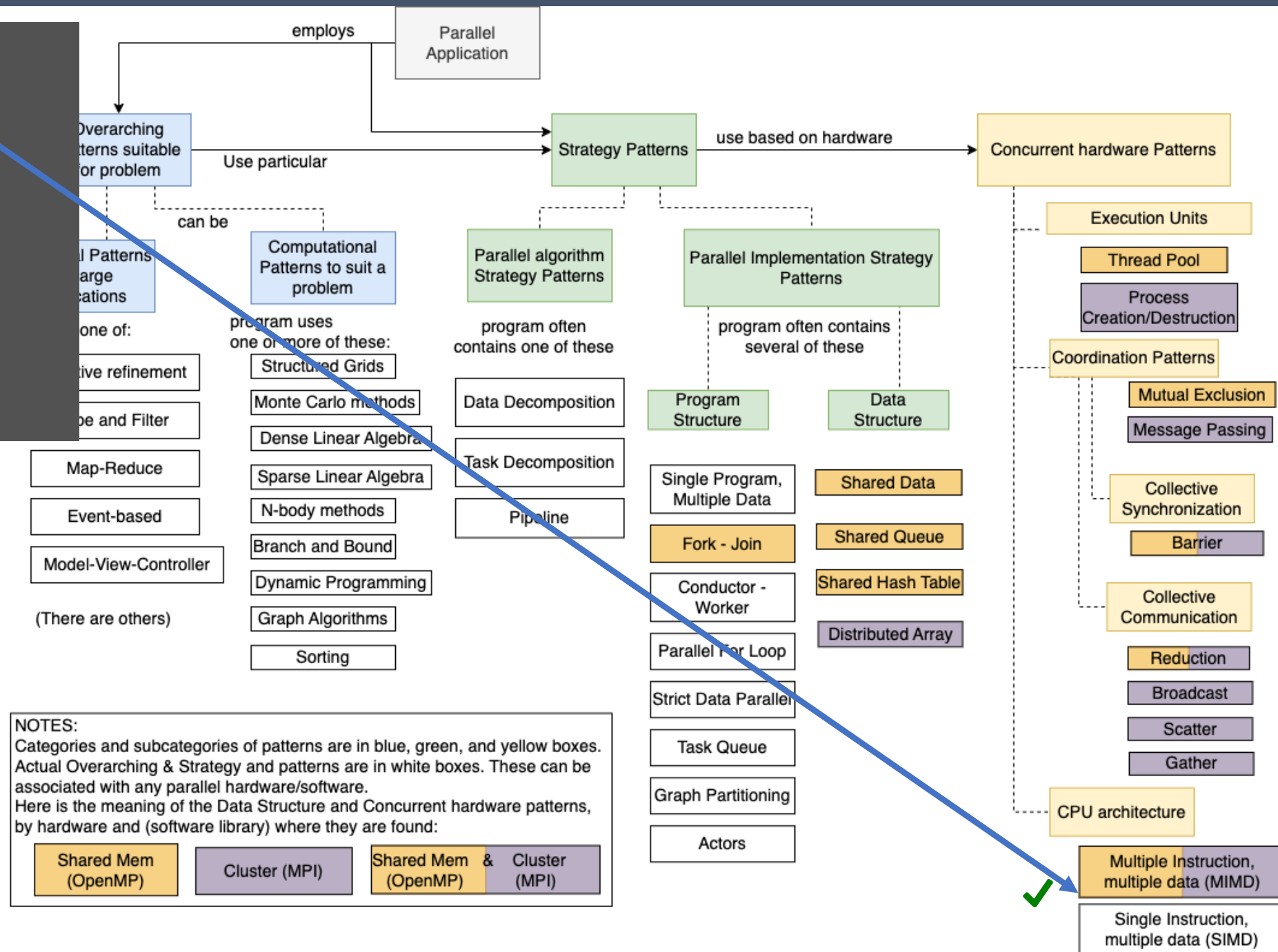
... //setup
#pragma omp parallel for \
default(none) \
private(i) \
shared(n, a, h) \
reduction(+:integral)
for(i = 1; i < n; i++) {
    integral += f(a+i*h);
}

integral = integral * h;

... // finish

```

OpenMP is based on systems where instructions run independently with no guarantee of order, each acting on its own shared or private data



```

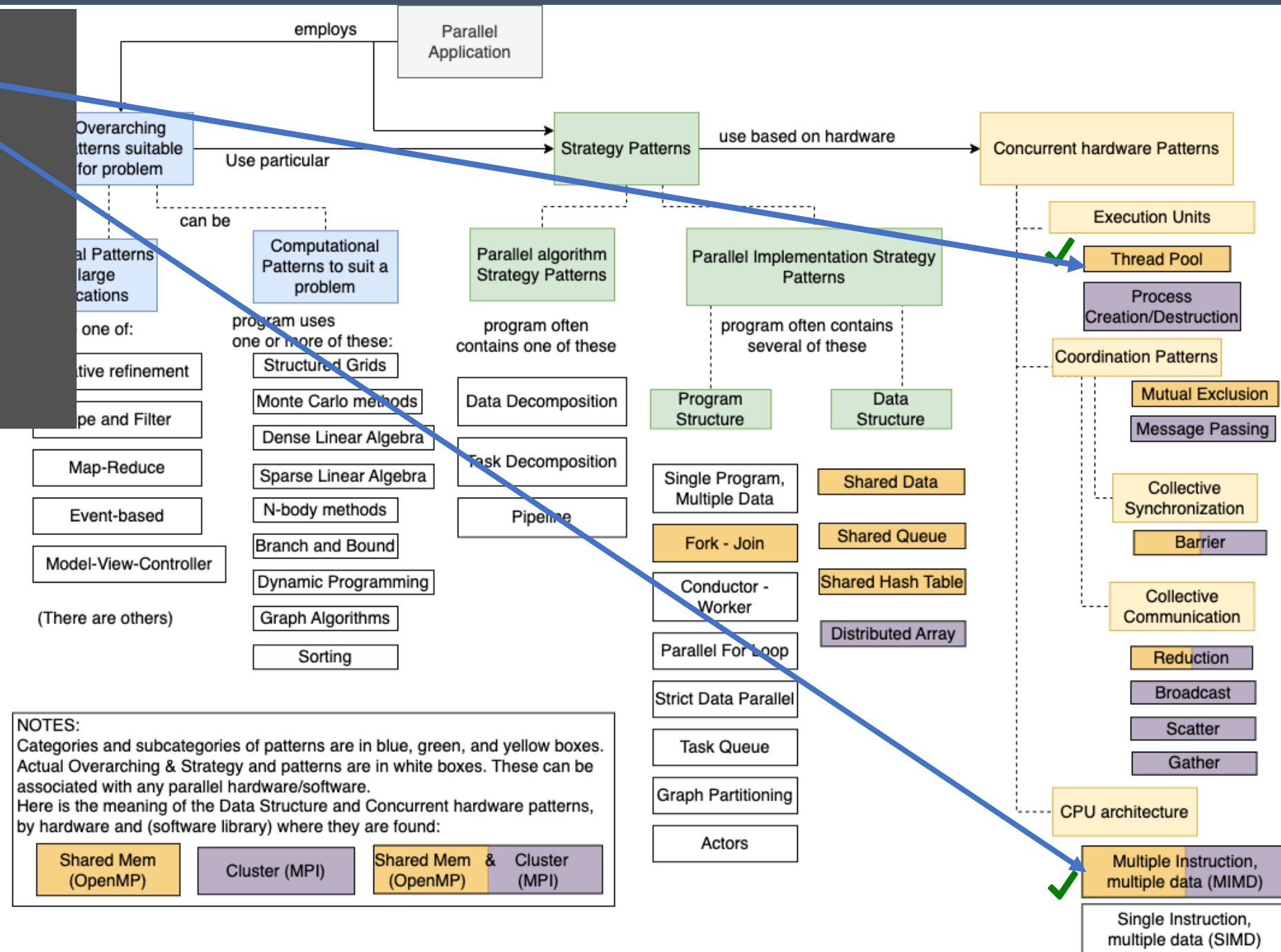
... //setup
#pragma omp parallel for \
default(none) \
private(i) \
shared(n, a, h) \
reduction(+:integral)
for(i = 1; i < n; i++) {
    integral += f(a+i*h);
}

integral = integral * h;

... // finish

```

OpenMP is implemented so that a group of threads is created as a 'pool' and used as needed

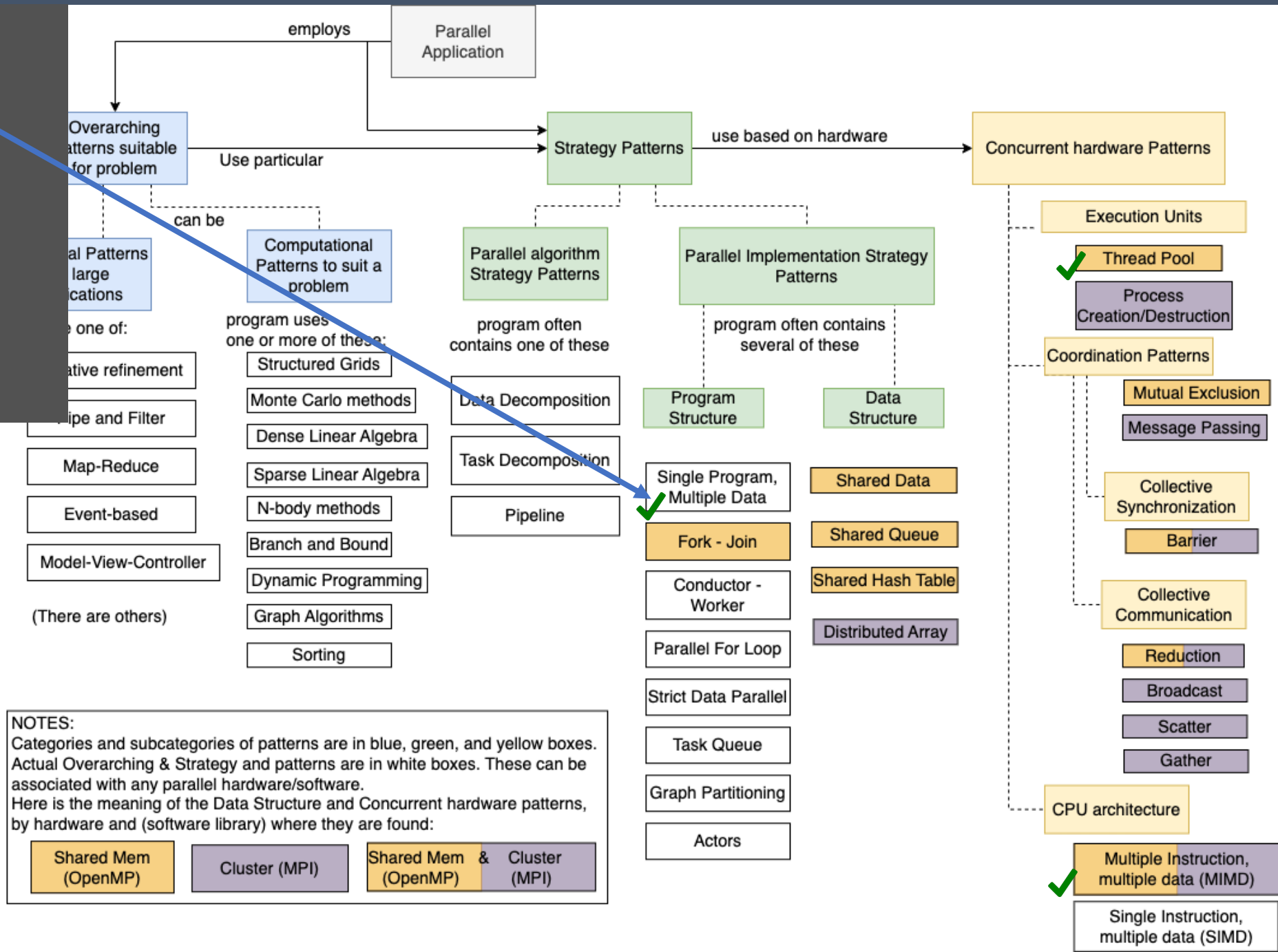


```
... //setup
#pragma omp parallel for \
default(none) \
private(i) \
shared(n, a, h) \
reduction(+:integral)
for(i = 1; i < n; i++) {
    integral += f(a+i*h);
}

integral = integral * h;

... // finish
```

OpenMP is implemented so that only one program is written




```

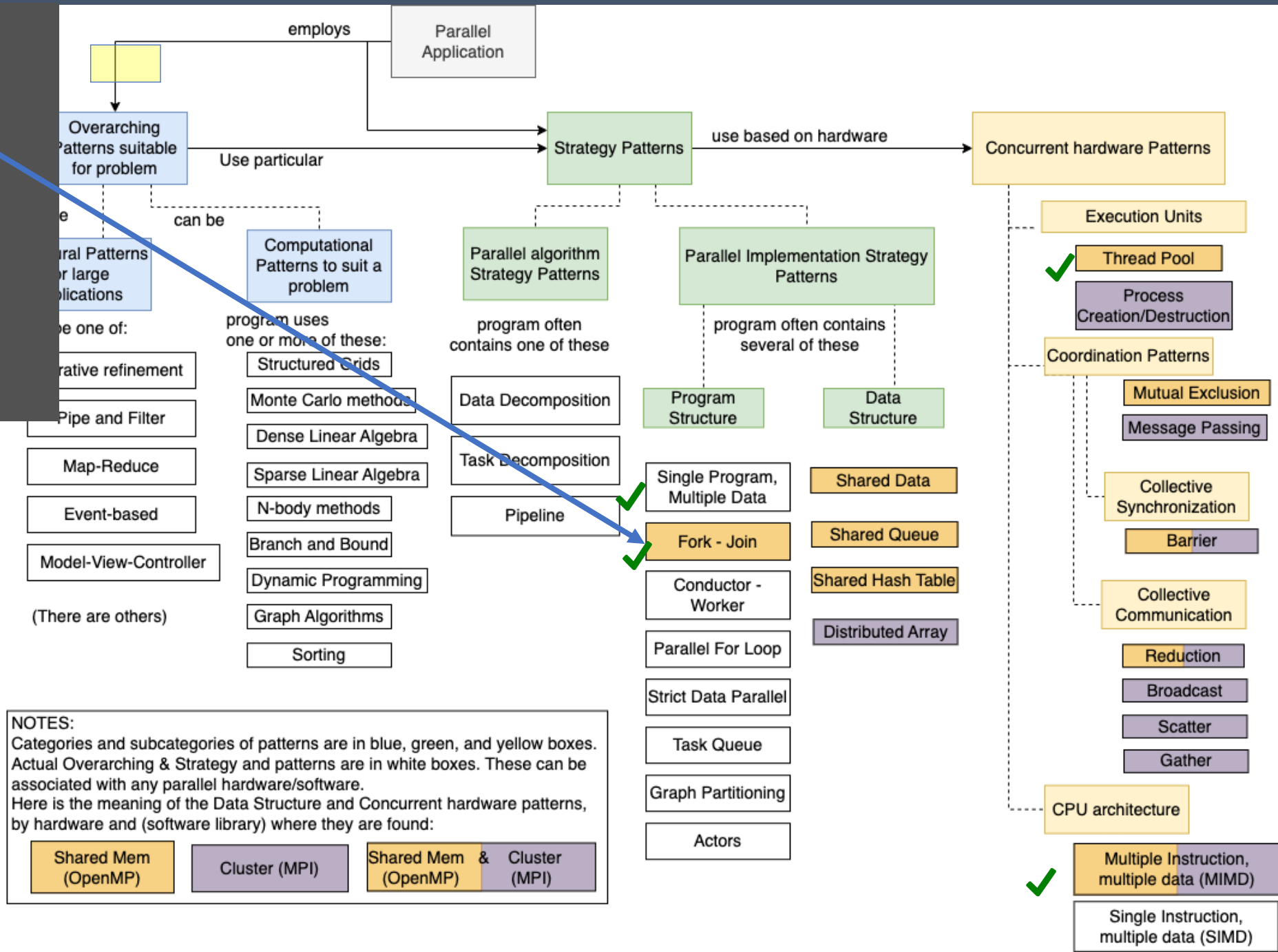
... //setup
#pragma omp parallel for \
default(none) \
private(i) \
shared(n, a, h) \
reduction(+:integral)
for(i = 1; i < n; i++) {
    integral += f(a+i*h);
}

integral = integral * h;

... // finish

```

OpenMP parallel keyword automatically forks threads and joins them when completed.



```

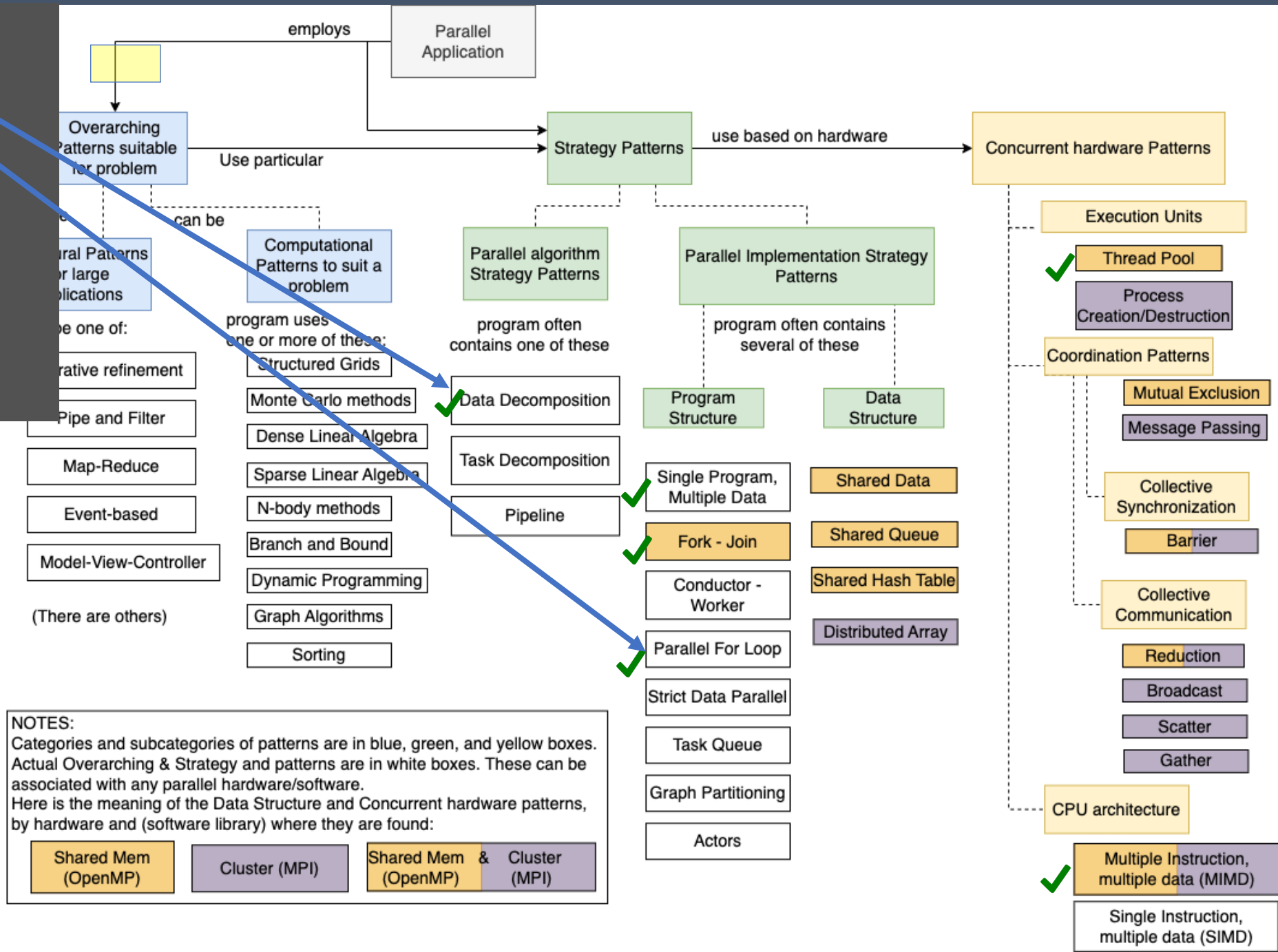
... //setup
#pragma omp parallel for \
default(none) \
private(i) \
shared(n, a, h) \
reduction(+:integral)
for(i = 1; i < n; i++) {
    integral += f(a+i*h);
}

integral = integral * h;

... // finish

```

OpenMP for keyword implies geometric decomposition of data among threads, using the parallel for loop.

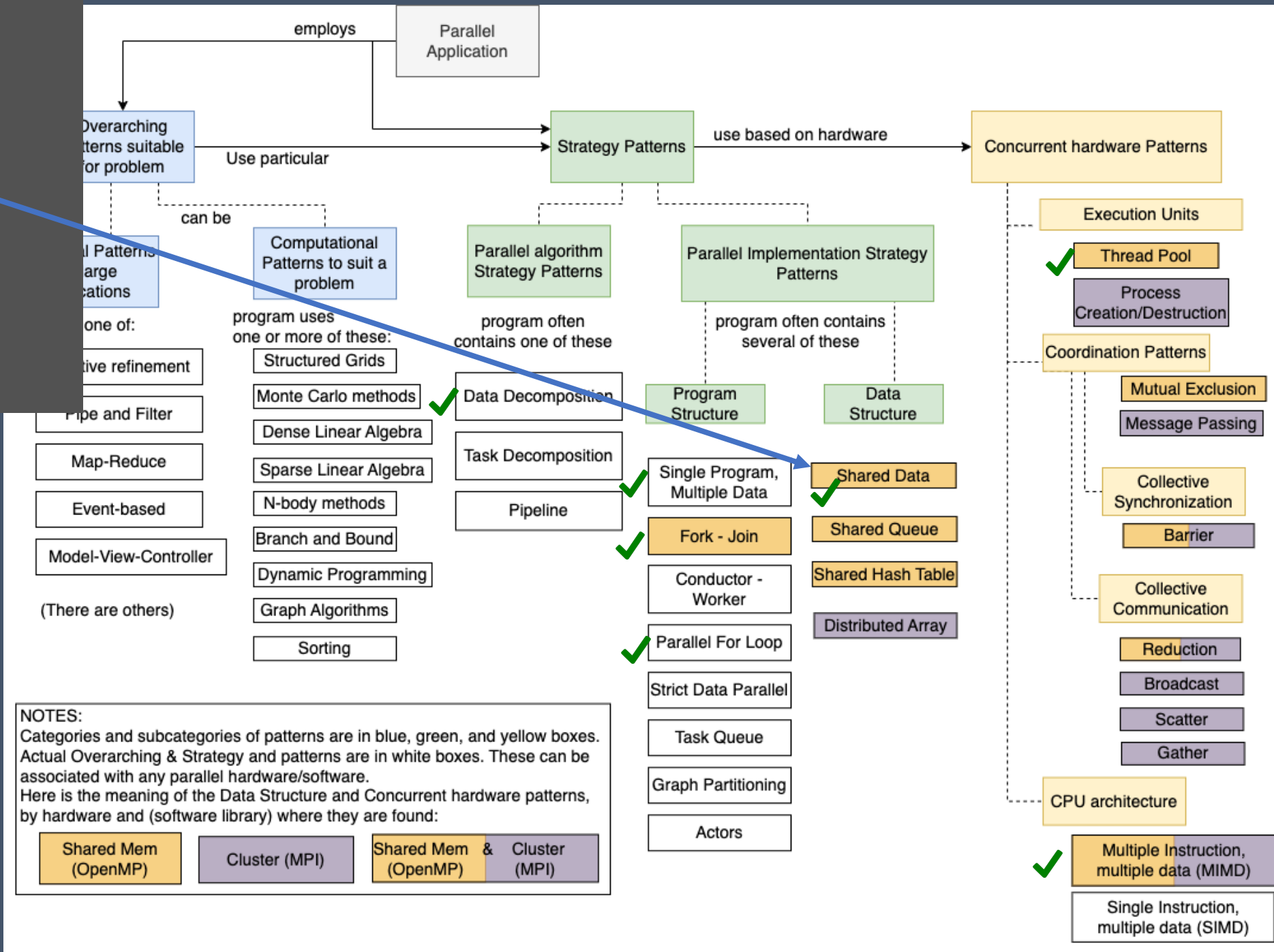



```
... //setup
#pragma omp parallel for \
default(none) \
private(i) \
shared(n, a, h) \
reduction(+:integral)
for(i = 1; i < n; i++) {
    integral += f(a+i*h);
}

integral = integral * h;

... // finish
```

Almost all OpenMP threaded programs use shared data



NOTES:
Categories and subcategories of patterns are in blue, green, and yellow boxes. Actual Overarching & Strategy and patterns are in white boxes. These can be associated with any parallel hardware/software.
Here is the meaning of the Data Structure and Concurrent hardware patterns, by hardware and (software library) where they are found:

Shared Mem (OpenMP)	Cluster (MPI)	Shared Mem & Cluster (OpenMP) (MPI)
---------------------	---------------	-------------------------------------

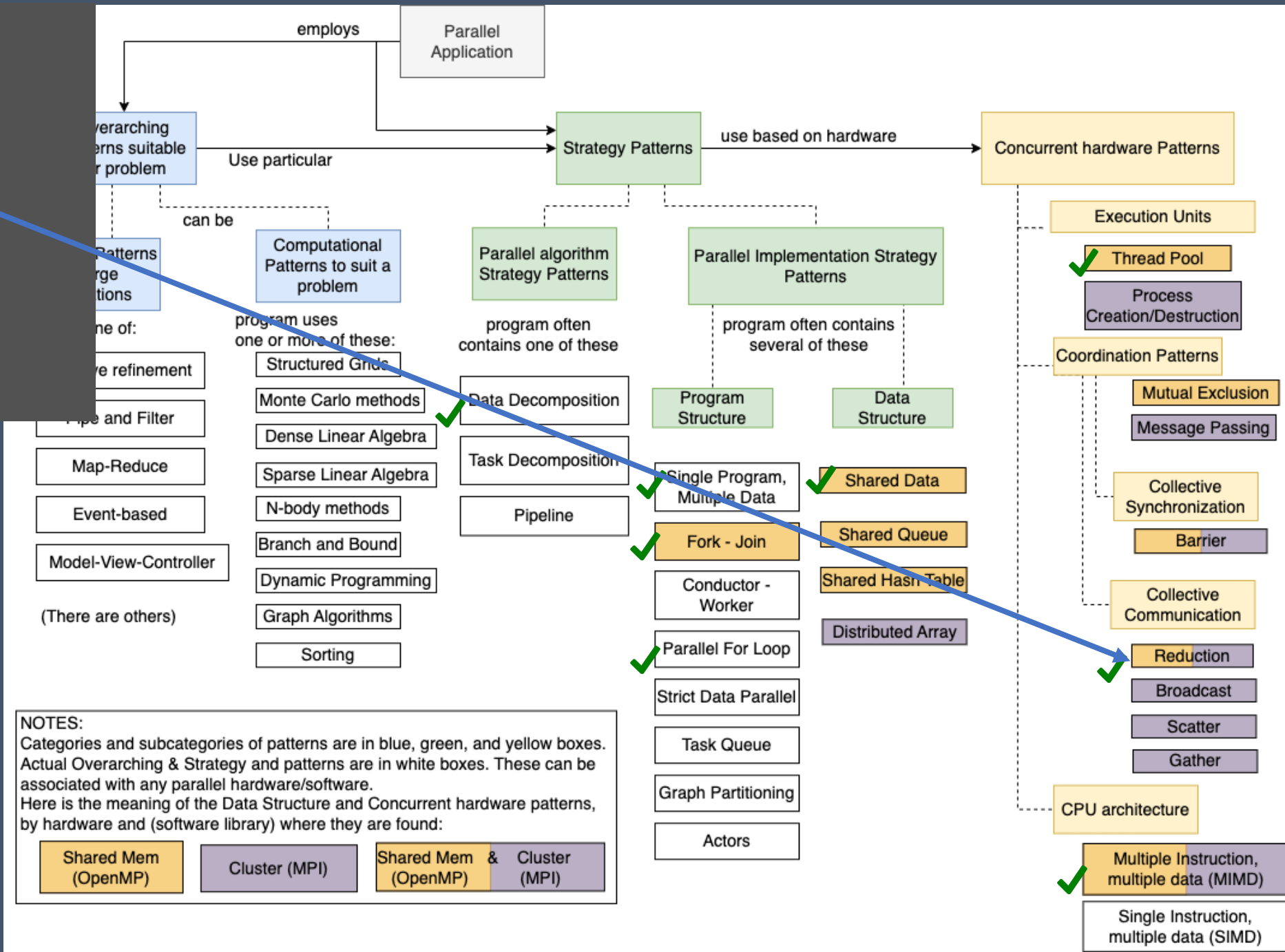
```

... //setup
#pragma omp parallel for \
default(none) \
private(i) \
shared(n, a, h) \
reduction(+:integral)
for(i = 1; i < n; i++) {
    integral += f(a+i*h);
}

integral = integral * h;

... // finish

```



Reduction is a common OpenMP pattern that is used to ensure threaded programs avoid using mutual exclusion to 'lock' a shared variable that each thread needs to update. Each thread keeps a private copy instead.

```

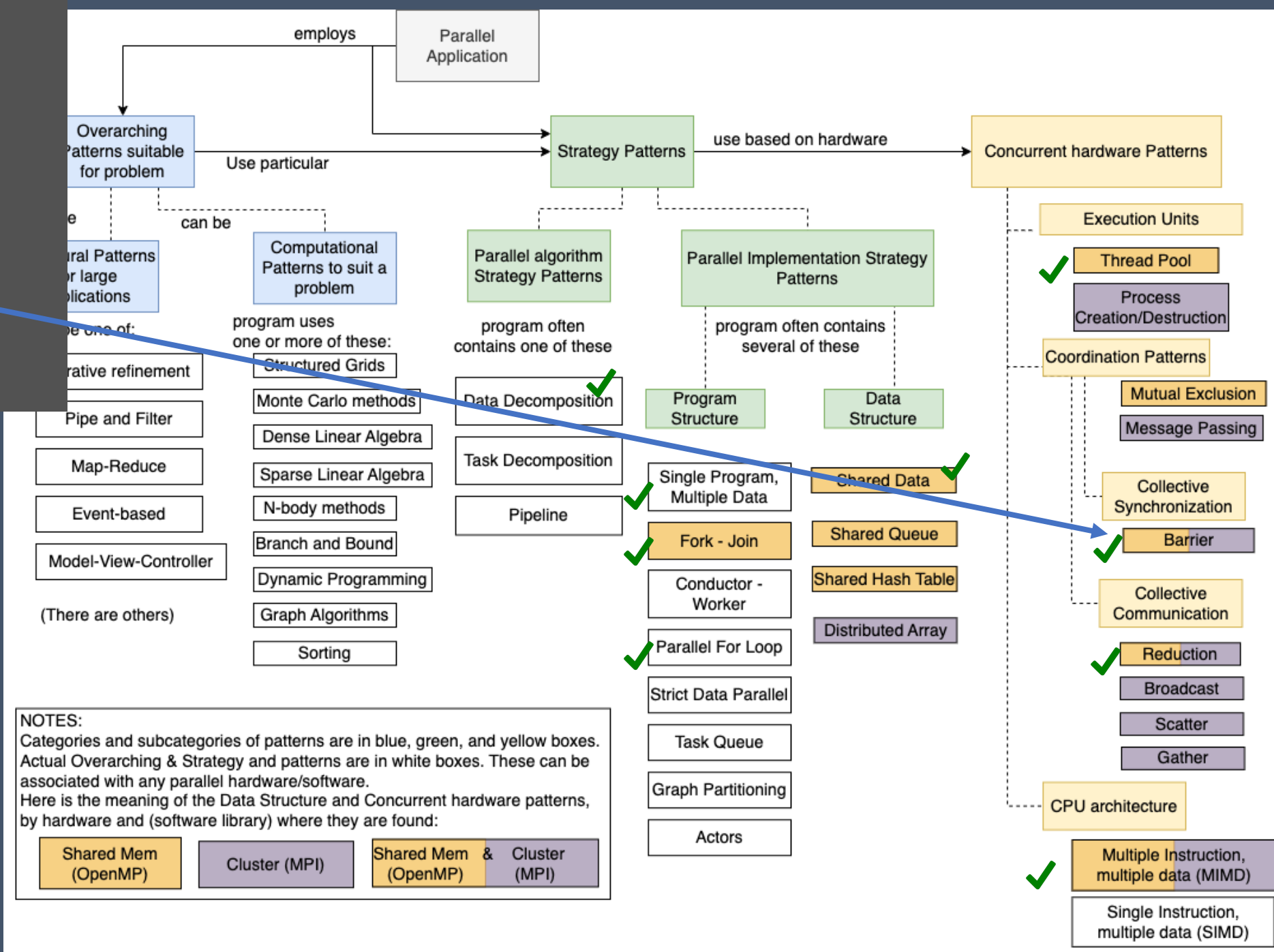
... //setup
#pragma omp parallel for \
default(none) \
private(i) \
shared(n, a, h) \
reduction(+:integral)
for(i = 1; i < n; i++) {
    integral += f(a+i*h);
}

integral = integral * h;

... // finish

```

At the end of every parallel for data decomposition, there is an implied 'barrier' used to ensure that all threads complete before the main thread continues.



```

... //setup
#pragma omp parallel for \
default(none) \
private(i) \
shared(n, a, h) \
reduction(+:integral)
for(i = 1; i < n; i++) {
    integral += f(a+i*h);
}

integral = integral * h;

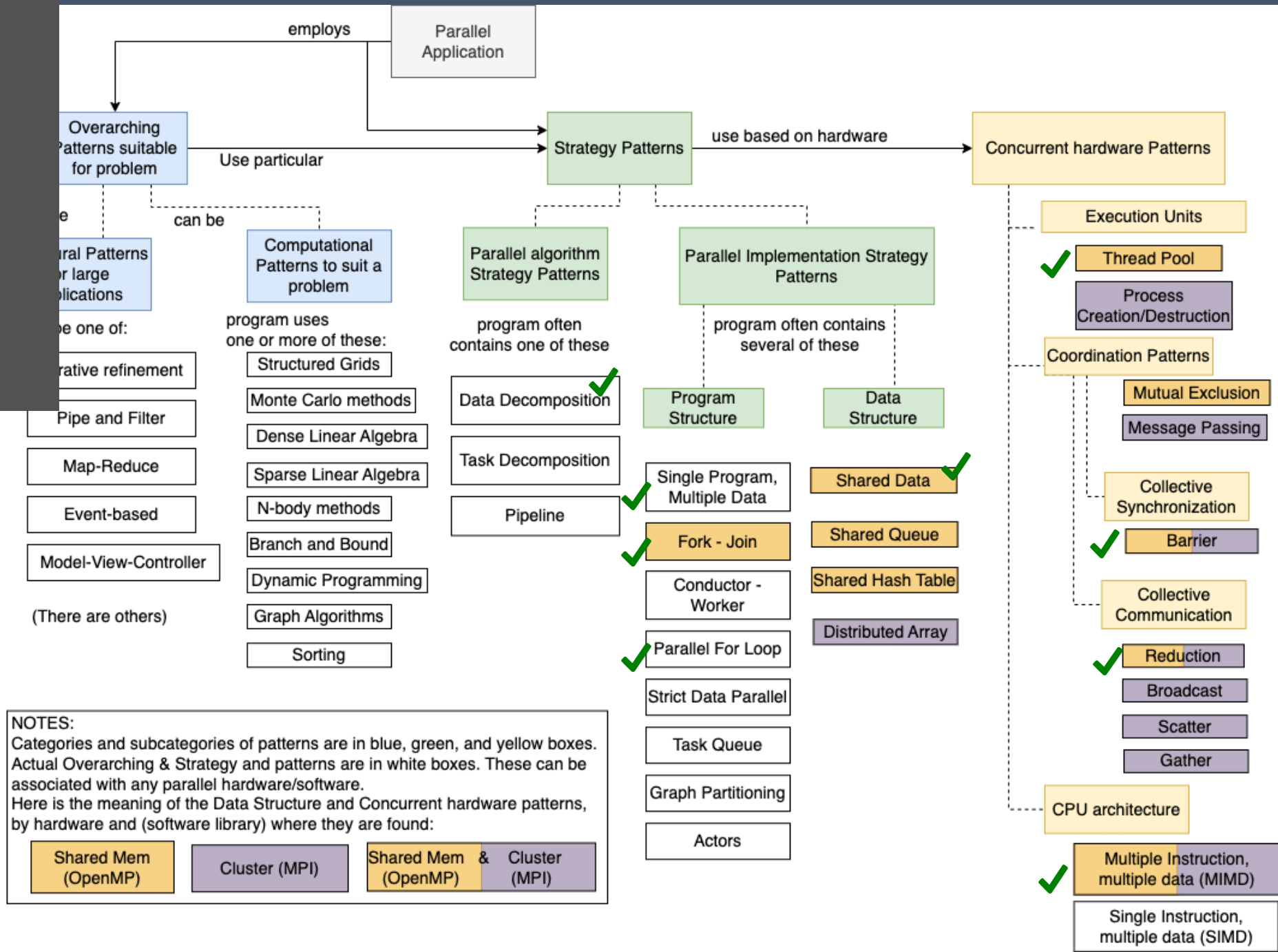
... // finish

```

trap-omp
example

Patterns found

Built into
OpenMP



trng random
demo
example

Patterns found

Some Built into
OpenMP

